

Testdoubles / Mockito Übungen

Calculator

```
class Calculator {  
  
    /**  
     * Verwende Mockito, um ein Mock-Objekt der Calculator-Klasse zu erstellen.  
     * Simuliere das Verhalten der Methode add(int a, int b), sodass sie immer 10 zurückgibt,  
     * egal welche Argumente übergeben werden.  
     * Schreibe einen Testfall, der überprüft, ob der Mock korrekt arbeitet.  
     */  
    int add(int a, int b) {  
        return a + b;  
    }  
}
```

Calculator: Lösung

```
import org.junit.jupiter.api.Test;

import static com.google.common.truth.Truth.assertThat;
import static org.mockito.ArgumentMatchers.anyInt;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.when;

class CalculatorTest {

    @Test
    void add() {
        Calculator calculatorMock = mock();

        // Hinweis:
        // Normalerweise würde man so einen Test nicht schreiben, da nicht nachvollziehbar.
        // Man würde mit konkreten Parametern arbeiten. Hier Argument Matcher nur zu Übungszwecke.
        when(calculatorMock.add(anyInt(), anyInt())).thenReturn(10);

        assertThat(calculatorMock.add(4, 6)).isEqualTo(10);
    }
}
```

UserService

```
class UserService {  
  
    private User storedUser;  
  
    /**  
     * Erstelle ein Mock-Objekt von UserService.  
     * Schreibe einen Test, der prüft, ob die Methode storeUser aufgerufen wurde,  
     * egal welche Argumente übergeben werden.  
     */  
    void storeUser(User user) {  
        if (user != null) {  
            storedUser = user;  
            System.out.println("Stored user " + user.name());  
        }  
    }  
  
    /**  
     * Erstelle ein Mock-Objekt von User und speichere ihn in ein ungemockten UserService.  
     * Schreibe einen Test, der prüft, ob der gemockte User im UserService gespeichert wurde und  
     * ob die Methode name() von dem gemockten User aufgerufen wurde.  
     */  
    User getStoredUser() {  
        return storedUser;  
    }  
}
```

```
record User(String name) {}
```

UserService: Lösung

```
import org.junit.jupiter.api.Test;

import static com.google.common.truth.Truth.assertThat;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.verify;

class UserServiceTest {

    @Test
    void storeUserIsCalled() {
        UserService userServiceMock = mock();

        userServiceMock.storeUser(new User("Elon"));

        verify(userServiceMock).storeUser(any());
    }

    @Test
    void getUserAndNameOfUserIsCalled() {
        UserService userService = new UserService();

        User userMock = mock();
        userService.storeUser(userMock);

        assertThat(userService.getStoredUser()).isEqualTo(userMock);
        verify(userMock).name();
    }
}
```

ProductService

```
class ProductService {  
  
    /**  
     * Erstelle ein Mock-Objekt von ProductService.  
     * Simuliere verschiedene Rückgabewerte für die Methode getProductDetails(String productId):  
     * 1. Wenn die Produkt-ID „123“ ist, gebe "Product found." zurück.  
     * 2. Für alle anderen IDs gebe "Product not found." zurück.  
     * Schreibe entsprechende Testfälle.  
     */  
    String getProductDetails(String productId) {  
        return "ProductDetails";  
    }  
}
```

ProductService: Lösung

```
import org.junit.jupiter.api.Test;

import static com.google.common.truth.Truth.assertThat;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.when;

class ProductServiceTest {

    @Test
    void getProductDetails() {
        ProductService productServiceMock = mock();
        when(productServiceMock.getProductDetails(anyString())).thenReturn("Product not found.");
        when(productServiceMock.getProductDetails("123")).thenReturn("Product found.");

        String found = productServiceMock.getProductDetails("123");
        assertThat(found).isEqualTo("Product found.");

        String notFound = productServiceMock.getProductDetails("00000");
        assertThat(notFound).isEqualTo("Product not found.");
    }
}
```

PaymentService

```
class PaymentService {  
  
    /**  
     * Erstelle ein Mock-Objekt von PaymentService.  
     * Simuliere, dass die Methode processPayment() eine PaymentException auslöst, wenn der Betrag gleich 1001 ist.  
     * Schreibe einen Test, der prüft, ob die Exception korrekt geworfen wird.  
     *  
     * Bonusaufgabe:  
     * Schreibe den Test so um, dass der Mock eine Exception wirft, sobald ein Betrag größer als 1000 übergeben wird.  
     * Tipp: Schau in die Dokumentation von Mockito.  
     */  
    boolean processPayment(String creditCardNumber, double amount) throws PaymentException {  
        return true;  
    }  
}
```

```
class PaymentException extends RuntimeException {  
    PaymentException(String message) {  
        super(message);  
    }  
}
```

PaymentService: Lösung

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.AdditionalMatchers.gt;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.when;

class PaymentServiceTest {

    @Test
    void processPaymentThrowsException() {
        PaymentService paymentServiceMock = mock();

        double forbiddenAmount = 1001; // Achtung: Der Typ muss double sein!
        when(paymentServiceMock.processPayment(anyString(), eq(forbiddenAmount))).thenThrow(new PaymentException("Maximum limit is 1000."));

        assertThrows(PaymentException.class, () -> paymentServiceMock.processPayment("123", forbiddenAmount));
    }

    @Test
    void shouldThrowPaymentExceptionForAmountsGreaterThan1000_Bonus() {
        PaymentService paymentServiceMock = mock();

        // Achtung: Der Typ muss double sein!
        when(paymentServiceMock.processPayment(anyString(), gt(1000.0))).thenThrow(new PaymentException("Maximum limit is 1000."));
        // when(paymentServiceMock.processPayment(anyString(), doubleThat(d -> d > 1000))).thenThrow(new PaymentException("Maximum limit is 1000."));

        int forbiddenAmount = 1001;
        assertThrows(PaymentException.class, () -> paymentServiceMock.processPayment("123", forbiddenAmount));
    }
}
```

OrderService

```
class OrderService {  
  
    /**  
     * Erstelle ein Mock-Objekt von OrderService.  
     * Simuliere einen Test, bei dem eine Bestellung aufgegeben wird.  
     * Prüfe, ob der placeOrder mit einem bestimmten Order-Object aufgerufen wurde.  
     */  
    public void placeOrder(Order order) {  
        // Irgendeine Bestelllogik (spielt keine Rolle)  
    }  
}
```

```
record Order(String product, int quantity) {}
```

OrderService: Lösung

```
import org.junit.jupiter.api.Test;

import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.verify;

class OrderServiceTest {

    @Test
    void placeOrder() {
        OrderService orderServiceMock = mock();

        var tesla = new Order("Tesla", 42);
        orderServiceMock.placeOrder(tesla);

        verify(orderServiceMock).placeOrder(tesla);
    }
}
```

NotificationService

```
/**
 * Verwende Mockito, um ein Mock-Objekt von NotificationService zu erstellen.
 * Übergebe dieses Mock-Objekt über den Konstruktor von OrderService (Dependency Injection).
 * Schreibe einen Test, der die Methode placeOrder aufruft und überprüft,
 * ob die sendNotification-Methode des NotificationService mit der korrekten Nachricht aufgerufen wurde.
 */
class OrderService {

    private NotificationService notificationService;

    // Dependency Injection via Constructor
    OrderService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    void placeOrder(String product) {
        System.out.println("Order placed for: " + product);
        notificationService.sendNotification("Order placed for " + product);
    }
}
```

```
class NotificationService {

    void sendNotification(String message) {
        System.out.println("Notification sent: " + message);
    }
}
```

NotificationService: Lösung

```
import org.junit.jupiter.api.Test;

import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.verify;

class OrderServiceTest {

    @Test
    void placeOrder() {
        NotificationService notificationServiceMock = mock();

        OrderService orderService = new OrderService(notificationServiceMock);
        String tesla = "Tesla";
        orderService.placeOrder(tesla);

        verify(notificationServiceMock).sendNotification("Order placed for " + tesla);
    }
}
```

Playstore (ohne Mockito)

Gegeben sind die folgenden Klassen und folgendes Interface. Schreibe einen **Unit Test mit 100% Testabdeckung** für die Klasse **Playstore**. Schreibe hierzu einen **Mock für PlaystoreDatabase ohne Mockito**.

```
class Playstore {  
  
    private final PlaystoreDatabase database;  
  
    Playstore(PlaystoreDatabase database) {  
        this.database = database;  
    }  
  
    Collection<PlaystoreGame> searchGame(String name) {  
        try {  
            return database.findGamesByName(name);  
        } catch (IOException e) {  
            return List.of();  
        }  
    }  
}
```

```
record PlaystoreGame(String name, int rating) {  
}
```

```
interface PlaystoreDatabase {  
    List<PlaystoreGame> findGamesByName(String name) throws IOException;  
}
```

Playstore (ohne Mockito): Lösung

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

import static com.google.common.truth.Truth.assertThat;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class PlaystoreTest {

    private static class PlaystoreDatabaseMock implements PlaystoreDatabase {
        @Override
        public List<PlaystoreGame> findGamesByName(String name) throws IOException {
            var games = new ArrayList<>(){
                List.of(
                    new PlaystoreGame("Dead Space", 4),
                    new PlaystoreGame("Baldurs Gate 3", 5),
                    new PlaystoreGame("Space Invaders", 2),
                    new PlaystoreGame("Jagged Alliance 3", 5),
                    new PlaystoreGame("Aliens: Dark Descent", 4),
                    new PlaystoreGame("Disco Elysium", 5)
                );
            };

            return games.stream()
                .filter(game -> game.name().toLowerCase().contains(name.toLowerCase()))
                .toList();
        }
    }

    private Playstore playstore;

    @BeforeEach
    void setup() {
        playstore = new Playstore(new PlaystoreDatabaseMock());
    }

    @Test
    void findNoGames() {
        // Arrange
        var emptyList = List.of();
        // Act
        var actual = playstore.searchGame("no game");
        // Assert
        assertEquals(emptyList, actual); // JUnit
        assertThat(playstore.searchGame("no game").isEmpty(), // Truth
    }
}
```

```
@Test
void findOneGame() {
    // Arrange
    var expected = new ArrayList<>(){
        List.of(
            new PlaystoreGame("Aliens: Dark Descent", 4)
        );
    };
    // Act
    var actual = playstore.searchGame("Alien");
    // Assert
    assertEquals(expected, actual); // JUnit
    assertThat(playstore.searchGame("Alien").containsExactlyElementsIn(expected); // Truth
}

@Test
void findMultipleGames() {
    // Arrange
    var expected = new ArrayList<>(){
        List.of(
            new PlaystoreGame("Dead Space", 4),
            new PlaystoreGame("Space Invaders", 2)
        );
    };
    // Act
    var actual = playstore.searchGame("space");
    // Assert
    assertEquals(expected, actual); // JUnit
    assertThat(playstore.searchGame("space").containsExactlyElementsIn(expected); // Truth
}

@Test
void findGamesThrowsIOException() {
    // Arrange
    var database = new PlaystoreDatabase() {
        @Override
        public List<PlaystoreGame> findGamesByName(String name) throws IOException {
            throw new IOException("No games found");
        }
    };
    playstore = new Playstore(database);
    var expected = List.of();
    // Act
    var actual = playstore.searchGame("anything");
    // Assert
    assertEquals(expected, actual); // JUnit
    assertThat(playstore.searchGame("anything").isEmpty(), // Truth
}

@Test
void findNullThrowsException() {
    assertThrows(NullPointerException.class, () -> playstore.searchGame(null));
}
}
```

Playstore (mit Mockito)

Schreibe einen neuen Unit Test für die Klasse Playstore (siehe vorherige Aufgabe).

Nutze dieses Mal das Framework Mockito, um die Klasse PlaystoreDatabase zu mocken.

Playstore (mit Mockito): Lösung

```
import org.junit.jupiter.api.Test;

import java.io.IOException;
import java.util.Collection;
import java.util.List;

import static com.google.common.truth.Truth.assertThat;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

class PlaystoreTest {

    @Test
    void findOneGame() throws IOException {
        // Arrange
        List<PlaystoreGame> expectedGames = List.of(new PlaystoreGame("Zelda", 10));
        PlaystoreDatabase mockedDatabase = mockedDatabaseWith("Zelda", expectedGames);
        Playstore playstore = new Playstore(mockedDatabase);

        // Act
        Collection<PlaystoreGame> actualGames = playstore.searchGame("Zelda");

        // Assert
        // Unnötiges verify, da der Aufruf von findGamesByName indirekt durch searchGame
        // mitgetestet wird.
        verify(mockedDatabase).findGamesByName("Zelda"); // unnötig
        assertEquals(expectedGames, actualGames); // JUnit
        assertThat(actualGames).isEqualTo(expectedGames); // Truth
    }

    @Test
    void findMultipleGames() {
        // Arrange + Beispiel mit Hilfsmethode
        var expectedGames = List.of(
            new PlaystoreGame("Zelda: Breath of the Wild", 9),
            new PlaystoreGame("Zelda: A Link to a Past", 10));
        var playstore = new Playstore(mockedDatabaseWith("Zelda", expectedGames));

        // Act
        var actualGames = playstore.searchGame("Zelda");

        // Assert
        assertEquals(expectedGames, actualGames); // JUnit
        assertThat(actualGames).isEqualTo(expectedGames); // Truth
    }
}
```

```
@Test
void returnEmptyCollectionWhenNoGameWasFound() {
    // Arrange + Beispiel mit Hilfsmethode
    List<PlaystoreGame> expectedGames = List.of();
    var playstore = new Playstore(mockedDatabaseWith("Zelda", expectedGames));

    // Act
    var actualGames = playstore.searchGame("Zelda");

    // Assert
    assertEquals(expectedGames, actualGames); // JUnit
    assertThat(actualGames).isEmpty(); // Truth
}

@Test
void returnEmptyCollectionWhenExceptionIsThrown() throws IOException {
    // Arrange
    PlaystoreDatabase mockedDatabase = mock();
    when(mockedDatabase.findGamesByName("Zelda")).thenReturn(new IOException("Should be caught."));
    var playstore = new Playstore(mockedDatabase);

    // Act
    var actualGames = playstore.searchGame("Zelda");

    // Assert
    assertEquals(List.of(), actualGames); // JUnit
    assertThat(actualGames).isEmpty(); // Truth
}

/**
 * Helper Method:
 * Returns a mocked {@link PlaystoreDatabase} that returns the given list of games when the given name is searched.
 */
private static PlaystoreDatabase mockedDatabaseWith(String searchedName, List<PlaystoreGame> gamesToReturn) {
    PlaystoreDatabase databaseMock = mock();
    try {
        when(databaseMock.findGamesByName(searchedName)).thenReturn(gamesToReturn);
    } catch (IOException e) {
        throw new RuntimeException("Should never happen.");
    }
    return databaseMock;
}
}
```

PlayerDamageCalculator

```
class PlayerDamageCalculator {  
  
    /**  
     * Verwende Mockito, um die statische Methode RandomUtil.randomNumber zu mocken.  
     * Schreibe einen Test für die Klasse PlayerDamageCalculator, der sicherstellt, dass die statische Methode  
     * RandomUtil.randomNumber aufgerufen wird und konstant 1 im Unit Test zurückgibt.  
     * (also keine Zufallszahl, da Zufallszahlen ungünstig als Erwartungswerte für Testfälle sind).  
     *  
     * Hinweis: Nutze mockStatic  
     *  
     * @param max: the maximum possible random number that may be generated (exclusive).  
     * @return a random number between 0 (inclusive) and max (exclusive).  
     */  
    int randomDamage(int max) {  
        return RandomUtil.randomNumber(max);  
    }  
}
```

```
import java.util.Random;  
  
class RandomUtil {  
  
    private static final int SEED = 42;  
  
    static int randomNumber(int max) {  
        return new Random(SEED).nextInt(max);  
    }  
}
```

PlayerDamageCalculator: Lösung

```
import org.junit.jupiter.api.Test;
import org.mockito.MockedStatic;

import static com.google.common.truth.Truth.assertThat;
import static org.mockito.ArgumentMatchers.anyInt;
import static org.mockito.Mockito.mockStatic;
import static org.mockito.Mockito.when;

class PlayerDamageCalculatorTest {

    @Test
    void calculateAreaOfSquare() {
        try (MockedStatic<RandomUtil> staticMock = mockStatic(RandomUtil.class)) {
            // Arrange
            when(RandomUtil.randomNumber(anyInt())).thenReturn(1);
            //      staticMock.when((() -> RandomUtil.randomNumber(anyInt())).thenReturn(1); // Alternative Schreibweise.
            PlayerDamageCalculator playerDamageCalculator = new PlayerDamageCalculator();

            // Act
            int actual = playerDamageCalculator.randomDamage(1_000_000_000);

            // Assert
            assertThat(actual).isEqualTo(1);
        }
    }
}
```

PlayerDamageCalculator – Dependency Injection

```
class PlayerDamageCalculator {  
  
    /**  
     * Überlege dir, wie du die Klasse PlayerDamageCalculator ohne Mockito testen kannst.  
     * Überarbeite die Klassen PlayerDamageCalculator / RandomUtil entsprechend.  
     * Eventuell benötigst du zusätzliche Klassen oder Interfaces? #hint  
     * Schreibe einen passenden Unit Test für PlayerDamageCalculator.  
     * Die Methode randomNumber soll weiterhin konstant 1 im Unit Test zurückgeben.  
     * (also keine Zufallszahl, da Zufallszahlen ungünstig als Erwartungswerte für Testfälle sind).  
     *  
     * Hinweis: Nutze KEIN mockStatic  
     *  
     * @param max: the maximum possible random number that may be generated (exclusive).  
     * @return a random number between 0 (inclusive) and max (exclusive).  
     */  
    int randomDamage(int max) {  
        return RandomUtil.randomNumber(max);  
    }  
}
```

```
import java.util.Random;  
  
class RandomUtil {  
  
    private static final int SEED = 42;  
  
    static int randomNumber(int max) {  
        return new Random(SEED).nextInt(max);  
    }  
}
```

PlayerDamageCalculator – Dependency Injection: Lösung

```
class PlayerDamageCalculator {  
  
    private final RandomGenerator randomGenerator;  
  
    PlayerDamageCalculator(RandomGenerator randomGenerator) {  
        this.randomGenerator = randomGenerator;  
    }  
  
    int randomDamage(int max) {  
        return randomGenerator.randomNumber(max);  
    }  
}
```

```
import java.util.Random;  
  
class RandomUtil implements RandomGenerator {  
  
    private static final int SEED = 42;  
  
    public int randomNumber(int max) {  
        return new Random(SEED).nextInt(max);  
    }  
}
```

```
@FunctionalInterface  
interface RandomGenerator {  
  
    int randomNumber(int max);  
}
```

```
import org.junit.jupiter.api.Test;  
  
import static com.google.common.truth.Truth.assertThat;  
import static org.junit.jupiter.api.Assertions.assertEquals;  
  
class PlayerDamageCalculatorTest {  
  
    @Test  
    void calculateAreaOfSquare() {  
        // Arrange  
        // Das Lambda implementiert das Interface RandomGenerator und  
        // wird anschließend in PlayerDamageCalculator injiziert:  
        PlayerDamageCalculator playerDamageCalculator = new PlayerDamageCalculator(_ -> 1);  
  
        // Act  
        int actual = playerDamageCalculator.randomDamage(1_000_000_000);  
  
        // Assert  
        assertEquals(1, actual); // JUnit  
        assertThat(actual).isEqualTo(1); // Truth  
    }  
}
```